Cours MVSI
Modélisation et Vérification des Systèmes Informatiques

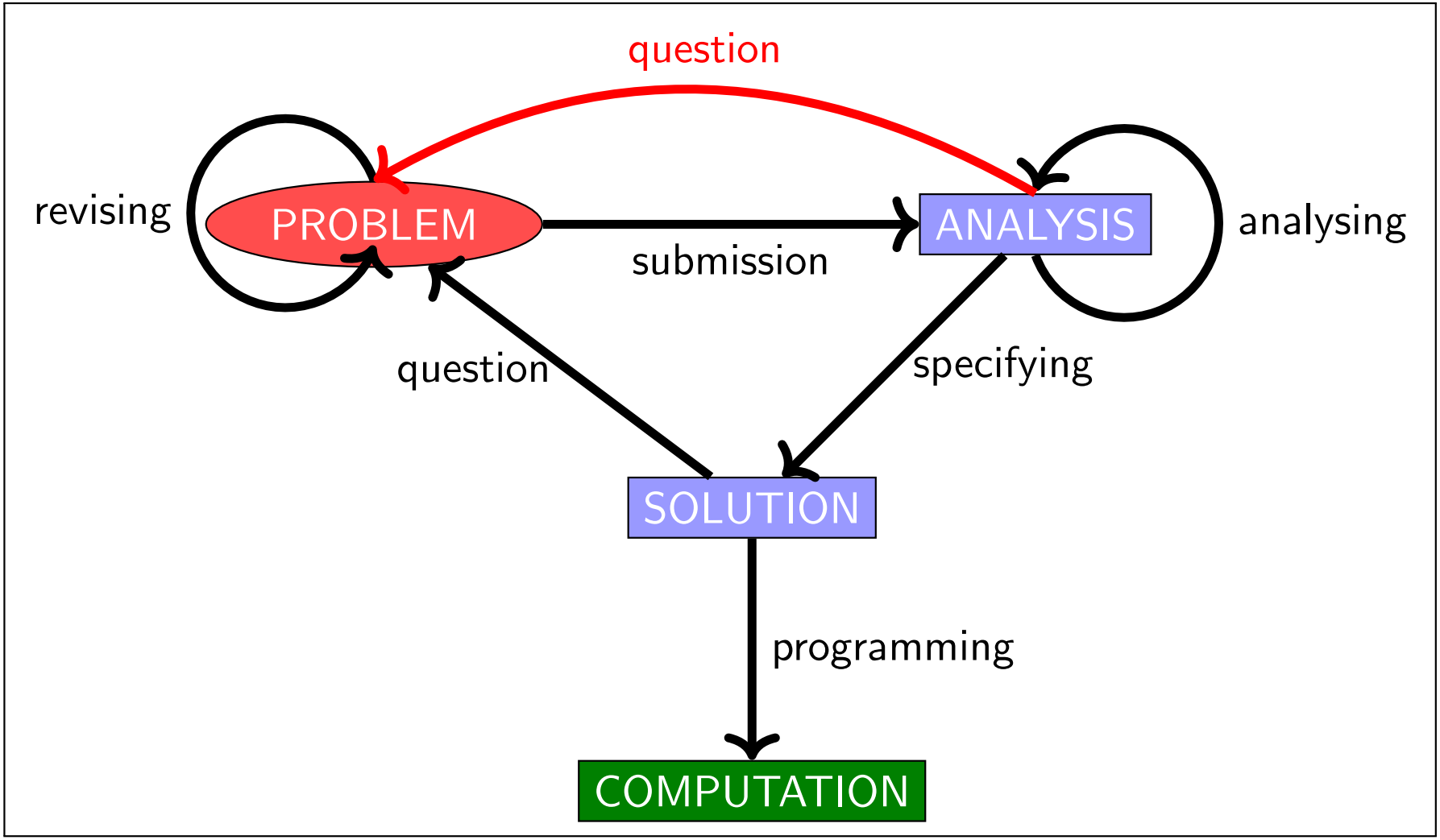
Sommaire du cours

Dominique Méry
Telecom Nancy, Université de Lorraine
(11 septembre 2026 at 12:24 A.M.)

Plan

1. Tracking bugs in C codes
2. Introduction by Example
   - Detecting overflows in computations
   - Computing the velocity of an aircraft on the ground
3. Verification of program properties
4. Topics of course

Problem versus Solution



Listing 1 – Bug bug0

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int x, y;
    // Seed the random number generator with the current time
    srand(time(NULL));
    // Generate a random number between 1 and 100
    x = rand() % 100 + 1;
    // Perform some calculations
    y = x / (100 - x);
    printf("x=%d--and-y=%d\n",x, y);
    return 0;
}
```

Listing 2 – Bug bug0

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int x, y;
    // Seed the random number generator with the current time
    srand(time(NULL));
    // Generate a random number between 1 and 100
    x = rand() % 100 + 1;
    // Perform some calculations
    y = x / (100 - x);
    printf("x=%d--and-y=%d\n",x, y);
    return 0;
}
```

bug0.c prints x=some number and y=some other number

Listing 3 – Bug bug00

```
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
//
int main() {
  int x, y,i=0;
    for (i = 0; i <= 1000000; i++) {
    // Seed the random number generator with the current time
    srand(time(NULL));
    // Generate a random number between 1 and 100
    x = rand() % 100 + 1;
        printf("Result: x=%d\n",x);
    // Perform some calculations
    y = x / (100 - x);
    printf("Result: i=%d and y=%d\n",i, y);
    }
    return 0;
}
```

Listing 4 – Bug bug00

```
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
//
int main() {
  int x, y,i=0;
    for (i = 0; i <= 1000000; i++) {
    // Seed the random number generator with the current time
    srand(time(NULL));
    // Generate a random number between 1 and 100
    x = rand() % 100 + 1;
        printf("Result:-x=--%d\n",x);
    // Perform some calculations
    y = x / (100 - x);
    printf("Result:-i=%d--and-y=%d\n",i, y);
    }
    return 0;
}
```

bug00.c prints ... i=—textitsome number and j=some other number ...

Listing 5 – Bug bug000

```
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>



int main() {
  int x, y,i=0;

    for (i = 0; i <= INT_MAX; i++) {
    // Seed the random number generator with the current time
    srand(time(NULL)+i);

    // Generate a random number between 1 and 100
     x = rand() % 100 + 1;

         printf("Result: x= %d\n",x);
    // Perform some calculations
    y = x / (100 - x);

    printf("Result: i=%d x=%d and y=%d\n",i,x, y);
    }

    return 0;
}
```

Listing 6 – Bug bug000

```c
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>



int main() {
  int x, y,i=0;

    for (i = 0; i <= INT_MAX; i++) {
    // Seed the random number generator with the current time
    srand(time(NULL)+i);

    // Generate a random number between 1 and 100
     x = rand() % 100 + 1;

        printf("Result: x= %d\n",x);
    // Perform some calculations
    y = x / (100 - x);

    printf("Result: i=%d x=%d and y=%d\n",i,x, y);
    }

    return 0;
}
```

Result: x= ??

Result: i=?? x=?? and y=??

RTE

- RTE means Run Time Error
- `x = rand() % 100 + 1` and `y = x / (100 - x)`
- $rte(\texttt{x = rand() \% 100 + 1}) \stackrel{def}{=} rte(\texttt{x}) \wedge rte(\texttt{rand()}) \wedge rte(\texttt{rand()\%100+1})$
- $rte(\texttt{y = x / (100 - x)}) \stackrel{def}{=} ret(\texttt{y}) \wedge rte(\texttt{x}) \wedge 100 \neq x$

RTE

The value of `x` may be *100* and a sequence of test can not get x with the value 100.

bug1.c

Listing 7 – Bug bug1

```c
#include <stdio.h>
#define n 5
int main() {
    int numbers[n] = {1, 2, 3, 4, 5};
    int sum = 0;

    // Attempt to calculate the sum of numbers in the array
    for (int i = 0; i <= n; i++) {
        sum += numbers[i];
    };
    int val = n*(n+1)/2;
    printf("Iteratively computed sum=%d and direct comutation val=%d\n", sum, val);
    return 0;
}
```

bug1.c

Listing 8 – Bug bug1

```c
#include <stdio.h>
#define n 5
int main() {
    int numbers[n] = {1, 2, 3, 4, 5};
    int sum = 0;

    // Attempt to calculate the sum of numbers in the array
    for (int i = 0; i <= n; i++) {
        sum += numbers[i];
    };
    int val = n*(n+1)/2;
    printf("Iteratively computed sum=%d and direct comutation val=%d\n", sum, val);
    return 0;
}
```

```
Iteratively computed sum=16 and
direct comutation val=15
```

Listing 9 – Bug bug2

```c
#include <stdio.h>

int main() {
    int x = 5;
    int y = 3;

    // Bug 1: Incorrect variable in the printf statement
    printf("The value of x is: %d\n", y);

    // Bug 2: Infinite loop
    while (x > 0) {
        printf("x is greater than 0\n");
    }

    return 0;
}
```

Listing 10 – Bug bug2

```c
#include <stdio.h>

int main() {
    int x = 5;
    int y = 3;

    // Bug 1: Incorrect variable in the printf statement
    printf("The value of x is: %d\n", y);

    // Bug 2: Infinite loop
    while (x > 0) {
        printf("x is greater than 0\n");
    }

    return 0;
}
```

Infinite loop . . .

Listing 11 – Bug bug7

```
#include <stdio.h>
#include <limits.h>
int average(int a,int b)
{
  return((a+b)/2);
}

int main()
{
  int x,y;
  x=INT_MAX;y=INT_MAX;
  printf("Average--for-%d-and-%d-is-%d\n",x,y,
         average(x,y));
  return 0;
}
```

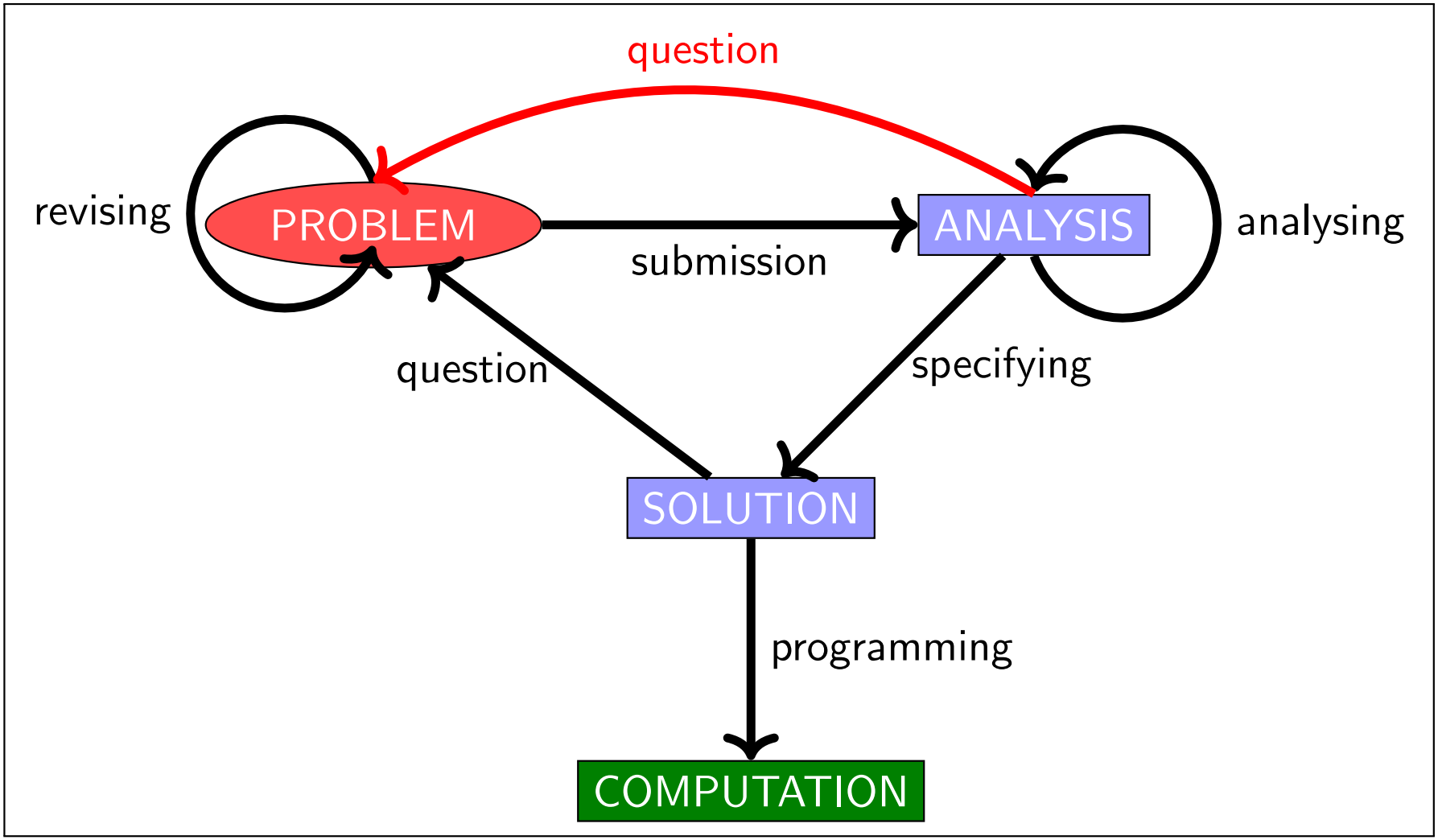
Listing 12 – Bug bug7

```
#include <stdio.h>
#include <limits.h>
int average(int a,int b)
{
  return((a+b)/2);
}

int main()
{
  int x,y;
  x=INT_MAX;y=INT_MAX;
  printf("Average--for-%d-and-%d-is-%d\n",x,y,
         average(x,y));
  return 0;
}
```

```
Average for 2147483647 and
2147483647 is -1
```

Problem versus Solution



Detecting overflows of computations

Listing 13 – Function average

```
#include <stdio.h>
#include <limits.h>
int average(int a,int b)
{
  return((a+b)/2);
}

int main()
{
  int x,y;
  x=INT_MAX;y=INT_MAX;
  printf("Average--for-%d-and-%d-is-%d\n",x,y,
         average(x,y));
  return 0;
}
```

Execution

Execution produces a result

```
Average  for 2147483647 and 2147483647 is -1
```

Execution

Execution produces a result

```
Average  for 2147483647 and 2147483647 is -1
```

Using frama-c produces a required annotation

```
int average(int a, int b)
{
  int __retres;
  /*@ assert rte: signed_overflow: -2147483648 <= a + b; */
  /*@ assert rte: signed_overflow: a + b <= 2147483647; */
  __retres = (a + b) / 2;
  return __retres;
}
```

Annotated Example 1

Listing 14 – Function average.....

```
#include <stdio.h>
#include <limits.h>
/*@  requires  0 <= a;
     requires a <= INT_MAX ;
     requires  0 <= b;
     requires b <= INT_MAX ;
     requires  0 <= a+b;
     requires a+b <= INT_MAX ;
     ensures \result <= INT_MAX;
*/
int average(int a,int b)
{
  return((a+b)/2);
}

int main()
{
  int x,y;
  x=INT_MAX / 2;y=INT_MAX / 2;
  //  printf("Average  for %d and %d is %d\n",x,y,
  //      );
  return average(x,y);
}
```

Nose Gear Velocity

- Estimated ground velocity of the aircraft should be available only if it is within 3 `km/hr` of the true velocity at some moment within past 3 seconds

Characterization of a System (I)

- NG velocity system :
  - **Hardware** :
    - *Electro-mechanical sensor* : detects rotations
    - *Two 16-bit counters* : Rotation counter, Milliseconds counter
    - *Interrupt service routine* : updates rotation counter and stores current time.
  - **Software** :
    - *Real-time operating system* : invokes update function every `500 ms`
    - *16-bit global variable* : for recording rotation counter update time
    - *An update function* : estimates ground velocity of the aircraft.
- Input data available to the system :
  - *time* : in `milliseconds`
  - *distance* : in `inches`
  - *rotation angle* : in `degrees`
- Specified system performs velocity estimations in *imperial* unit system
- Note : expressed functional requirement is in *SI* unit system (km/hr).

Characterization of a System (II) cont.

What are the main properties to consider for formalization ?

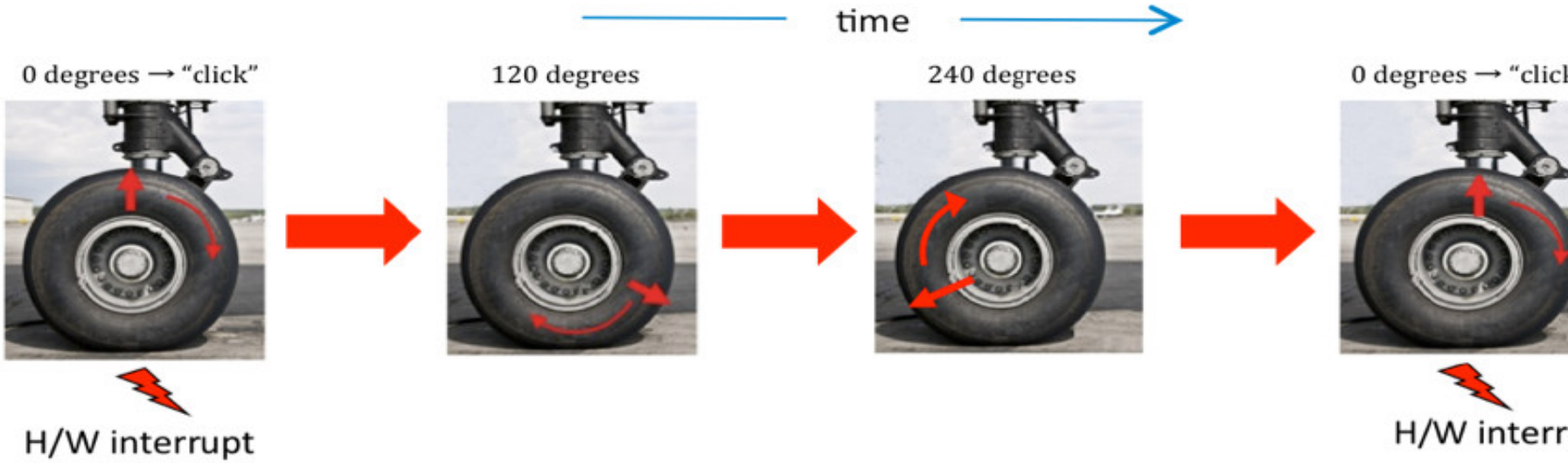
- Two different types of data :
  - counters with modulo semantics
  - non-negative values for time, distance, and velocity
- Two dimensions : *distance* and *time*
- Many units : distance (`inches`, `kilometers`, `miles`), time (`milliseconds`, `hours`), velocity (`kph`, `mph`)
- And interaction among components

How should we model ?

- Designer needs to consider units and conversions between them to manipulate the model
- One approach : Model units as *sets*, and conversions as constructed types – *projections*.
- Example :

1 $estimateVelocity \in \texttt{MILES} \times \texttt{HOURS} \rightarrow \texttt{MPH}$
2 $mphTokph \in \texttt{MPH} \rightarrowtail\!\!\!\!\!\rightarrow \texttt{KPH}$

Sample Velocity Estimation



NGClickTime = 4123 millisecs
NGRotations = 8954

NGClickTime = 4367milli
NGRotations = 8955

WHEEL_DIAMETER = 22 inches
PI = 3.14

12 inches/foot
5280 feet/mile

estimatedGroundVelocity = distance travel/elapsed time
= ((3.14 * 22)/(12*5280))/((4367-4123)/(1000*3600
= 16 mph

Safety Property

- Storing the number of `NGClick` in a n-bit variable VNGClick
- Integers are denoted by the set *Int* and is simply defined by the interval $Int \hat{=} INT_MIN..INT_MAX$.
- Safety requirement :
  The value of `VNGClick` is always in the range of implementation Int
  or equivalently $VNGClick \in Int$

- $Length = \pi \cdot diameter \cdot VNGClick$ (mathematical property)
- $Length \leq 6000$ (domain property)
- $\pi \cdot diameter \cdot VNGClick \leq 6000$
- $VNGClick \leq 6000/(\pi \cdot diameter)$
- if n=8, then $2^7 - 1 = 127$ and $6000/(\pi \cdot [22, inch]) = 6000/(\pi \cdot 55,88) = 6000/(3,24 \cdot [55,88, cm]) = 6000/(3,24 \cdot 0.5588) \approx 3419$ and the condition of safety can not be satisfied in any situation.
- if n=16, then $2^{15} - 1 = 65535$ and $6000/(\pi \cdot [22, inch]) \approx 3419$ and the condition of safety can be satisfied in any situation since $3419 \leq 65535$

Safety Property

- Storing the number of `NGClick` in a n-bit variable VNGClick
- Integers are denoted by the set *Int* and is simply defined by the interval *Int*$\hat{=}INT_MIN..INT_MAX$.
- Safety requirement :
  The value of `VNGClick` is always in the range of implementation Int
  or equivalently $VNGClick \in Int$

$$RTE_VNGClick : 0 \leq vNGClick \leq INT_MAX \quad (1)$$

- The current value of VNGClick is always bounded by the two values 0 and INT_MAX.

Verification and Validation

- Validation : *Are we building the right product*
- Verification : *Are we building the process right ?*

verification

The verification aims to check that the software meets its stated functional and non-functional requirements.

- *functional requirements*
- *non-functional requirements*

validation

The verification aims to ensure that the software meets the customer's expectations.

Verification of program properties

- Typing Properties using Typechecker (see for instance functional programming languages as ML, CAML, OCAML, ...)
- Invariance and safety (A *nothing bad will happen !*) properties for a program P :
  - Transformation of P into a relational model M simulating P
  - Expression of safety properties :
    $\forall s, s' \in \Sigma.(s \in Init_{\mathrm{S}} \wedge s \xrightarrow{\star} s') \Rightarrow (s' \in A)$.
  - Definition of the set of reachable states of P using M :
    $\textsc{reachable}(\mathrm{M}) = Init_{\mathrm{S}} \cup \longrightarrow [\textsc{reachable}(\mathrm{M})]$
  - Main property of $\textsc{reachable}(\mathrm{M})$: $\textsc{reachable}(\mathrm{M}) \subseteq A$
  - Characterization of $\textsc{reachable}(\mathrm{M})$:
    $\textsc{reachable}(\mathrm{M}) = \mathrm{FP}(\textsc{reachable}(\mathrm{M}))$

Problems

- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$:

Problems

- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !

Problems

- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !
- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$:

Problems

- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !
- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: possible when restrictions over the set of states is possible (finite set of states)

Problems

- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !
- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: possible when restrictions over the set of states is possible (finite set of states)
- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$:

Problems

- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !
- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: possible when restrictions over the set of states is possible (finite set of states)
- Proving automatically $\textsc{reachable}(\mathrm{M}) \subseteq A$: possible for some classes of systems and with some tools.

Problems

- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !
- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: possible when restrictions over the set of states is possible (finite set of states)
- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: possible for some classes of systems and with some tools.
- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: changing the domain and solving in another domain as abstract interpretation if making possible

Problems

- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !
- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: possible when restrictions over the set of states is possible (finite set of states)
- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: possible for some classes of systems and with some tools.
- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$: changing the domain and solving in another domain as abstract interpretation if making possible
- Proving automatically $\text{REACHABLE}(\mathrm{M}) \subseteq A$:

Problems

- Proving automatically $\text{REACHABLE}(\text{M}) \subseteq A$: undecidable . . . no program is able to prove it automatically !
- Proving automatically $\text{REACHABLE}(\text{M}) \subseteq A$: possible when restrictions over the set of states is possible (finite set of states)
- Proving automatically $\text{REACHABLE}(\text{M}) \subseteq A$: possible for some classes of systems and with some tools.
- Proving automatically $\text{REACHABLE}(\text{M}) \subseteq A$: changing the domain and solving in another domain as abstract interpretation if making possible
- Proving automatically $\text{REACHABLE}(\text{M}) \subseteq A$: approximating semantics of programs

Decidability or Undecidability

Decidability or Undecidability

- A problem $x \in P$ is generally stated by the function $\chi_{x \in P}$ where $\chi_{x \in P}(u) = 1$, if $P(u)$ is true and $\chi_{x \in P}(u) = 0$, if $P(u)$ is false :
  - Problem 1 : $x \in 0..n$ where $n \in \mathbb{N}$
  - Problem 1 : $w \in \mathcal{L}(G)$ where G is a grammar over the finite set of alphabet symbols Σ and $\mathcal{L}(G) \subseteq \Sigma^\star$.

Decidability or Undecidability

- A problem $x \in P$ is generally stated by the function $\chi_{x \in P}$ where $\chi_{x \in P}(u) = 1$, if $P(u)$ is true and $\chi_{x \in P}(u) = 0$, if $P(u)$ is false :
  - Problem 1 : $x \in 0..n$ where $n \in \mathbb{N}$
  - Problem 1 : $w \in \mathcal{L}(G)$ where G is a grammar over the finite set of alphabet symbols Σ and $\mathcal{L}(G) \subseteq \Sigma^\star$.
- A problem $x \in P$ is decidable, when the function $\chi_{x \in P}$ is computable or more precisely the function can be computed by a program

Decidability or Undecidability

- A problem $x \in P$ is generally stated by the function $\chi_{x \in P}$ where $\chi_{x \in P}(u) = 1$, if $P(u)$ is true and $\chi_{x \in P}(u) = 0$, if $P(u)$ is false :
  - Problem 1 : $x \in 0..n$ where $n \in \mathbb{N}$
  - Problem 1 : $w \in \mathcal{L}(G)$ where G is a grammar over the finite set of alphabet symbols Σ and $\mathcal{L}(G) \subseteq \Sigma^\star$.
- A problem $x \in P$ is decidable, when the function $\chi_{x \in P}$ is computable or more precisely the function can be computed by a program
- Problem of the correctness of a program :
  - Assume that $\mathcal{F}$ is the set of unary function over natural numbers : $\mathcal{F} = \mathbb{N} \nrightarrow \mathbb{N}$.
  - $\mathcal{C} \subseteq \mathcal{F}$: the set of computable (or programmable) functions is $\mathcal{C}$
  - $f \in \mathcal{C} = \{\Phi_0, \Phi_1, \ldots, \Phi_n, \ldots\}$: the set of computable functions is denumerable.
  - The problem $x \in dom(\Phi_y)$ is not decidable and it expresses the correctness of programs.

Implicite versus explicite

- Ecrire $101 = 5$ peut avoir une signification

Implicite versus explicite

- Ecrire *101 = 5* peut avoir une signification
- Le code du nombre n est *101* à gauche du symbole $=$ et le code du nombre n est sa représentation en base 10 à droite.
- $n_{10} = 5$ et $n_2 = 101$
- Vérification : $base(2, 10, 101) = 1.2^2 + 0.2 + 1.2^0 = 5_{10}$

Example : description of static behaviour

- A train moving at absolute speed $spd1$
- A person walking in this train with relative speed $spd2$
  - One may compute the absolute speed of the person
- Modelling
  - Syntax. Classical expressions
    - Type $Speed = Float$
    - $spd1, spd2 : Speed$
    - $AbsoluteSpeed = spd1 + spd2$
  - Semantics
    - If $spd1 = 25.6$ and $spd2 = 24.4$ then $AbsoluteSpeed = 50.0$
    - If $spd1 =$ "val" and $spd2 = 24.4$ then exception raised
  - Pragmatics
    - What if $spd1 is$ given in mph (miles per hour) and $spd2$ in km/s (kilometers per second) ?
    - What if $spd1$ is a relative speed ?

Intuition

- Un programme P *produit* des résultats à partir de données en accord avec une sémantique :
  - STATES est l'ensemble de tous les états de P : $STATES = \mathrm{x} \rightarrow \mathbb{Z}$ où x désigne les variables de P.
  - s_0 et s_f deux états de STATES : $\mathcal{D}(P)(s_0) = s_f$ signifie que P est exécuté à partir d'un état s_0 et produit un état s_f.
  - Pour un état s de P courant, on notera $s(\mathrm{x}) = x$ pour distinguer la valeur de la variable x et sa valeur courante en s :

Intuition

- Un programme P *produit* des résultats à partir de données en accord avec une sémantique :
  - STATES est l'ensemble de tous les états de P : $STATES = \mathrm{x} \rightarrow \mathbb{Z}$ où x désigne les variables de P.
  - s_0 et s_f deux états de STATES : $\mathcal{D}(P)(s_0) = s_f$ signifie que P est exécuté à partir d'un état s_0 et produit un état s_f.
  - Pour un état s de P courant, on notera $s(\mathrm{x}) = x$ pour distinguer la valeur de la variable x et sa valeur courante en s :

$$s_0(\mathrm{x}) = x_0,\ s_f(\mathrm{x}) = x_f,\ s'(\mathrm{x}) = x'$$

Intuition

- Un programme P *produit* des résultats à partir de données en accord avec une sémantique :
  - STATES est l'ensemble de tous les états de P : $STATES = \mathrm{x} \rightarrow \mathbb{Z}$ où x désigne les variables de P.
  - s_0 et s_f deux états de STATES : $\mathcal{D}(P)(s_0) = s_f$ signifie que P est exécuté à partir d'un état s_0 et produit un état s_f.
  - Pour un état s de P courant, on notera $s(\mathrm{x}) = x$ pour distinguer la valeur de la variable x et sa valeur courante en s :
    $$s_0(\mathrm{x}) = x_0,\ s_f(\mathrm{x}) = x_f,\ s'(\mathrm{x}) = x'$$
  - $\mathcal{D}(P)(s_0) = s_f$ définit la relation suivante sur l'ensemble des valeurs :
    $$x_0 \xrightarrow{\mathrm{P}} x_f$$

Intuition

- Un programme P *produit* des résultats à partir de données en accord avec une sémantique :
  - STATES est l'ensemble de tous les états de P : $STATES = \mathrm{x} \rightarrow \mathbb{Z}$ où x désigne les variables de P.
  - s_0 et s_f deux états de STATES : $\mathcal{D}(P)(s_0) = s_f$ signifie que P est exécuté à partir d'un état s_0 et produit un état s_f.
  - Pour un état s de P courant, on notera $s(\mathrm{x}) = x$ pour distinguer la valeur de la variable x et sa valeur courante en s :
    $$s_0(\mathrm{x}) = x_0,\ s_f(\mathrm{x}) = x_f,\ s'(\mathrm{x}) = x'$$
  - $\mathcal{D}(P)(s_0) = s_f$ définit la relation suivante sur l'ensemble des valeurs :
    $$x_0 \xrightarrow{\mathrm{P}} x_f$$
- Un programme P *remplit* un contrat (pre,post) :
  - P transforme une variable x à partir d'une valeur initiale x_0 et produisant une valeur finale x_f : $x_0 \xrightarrow{\mathrm{P}} x_f$
  - x_0 satisfait pre : $\mathrm{pre}(x_0)$
  - x_f satisfait post : $\mathrm{post}(x_0, x_f)$
  - $\mathrm{pre}(x_0) \wedge x_0 \xrightarrow{\mathrm{P}} x_f \Rightarrow \mathrm{post}(x_0, x_f)$

Vérification du contrat

Un programme P *remplit* un contrat (pre,post) :

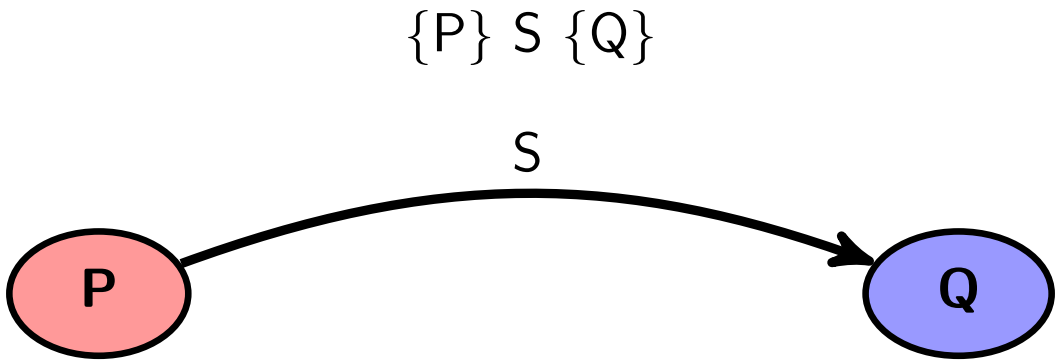
- P transforme une variable x à partir d'une valeur initiale x_0 et produisant une valeur finale x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : pre(x_0) and x_f satisfait post : post(x_0, x_f)
- pre$(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow$ post(x_0, x_f)

requires $pre(x_0)$
ensures $post(x_0, x_f)$
variables X

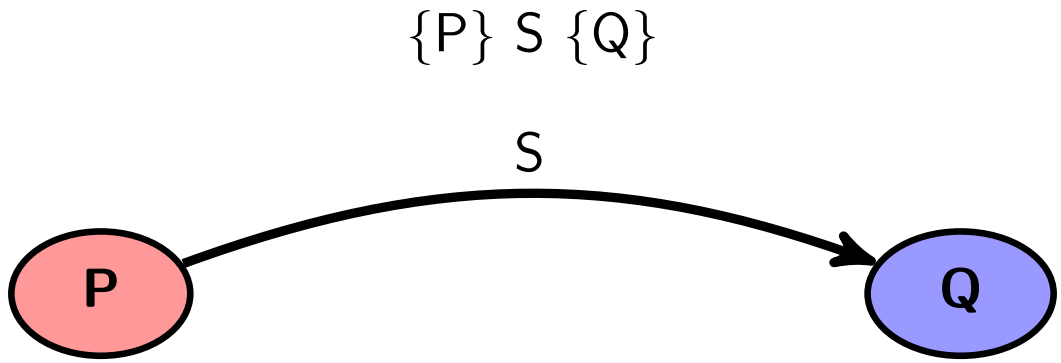
begin
$0 : P_0(x_0, x)$
$\texttt{instruction}_0$
$\ldots$
$i : P_i(x_0, x)$
$\ldots$
$\texttt{instruction}_{f-1}$
$f : P_f(x_0, x)$
end

- $pre(x_0) \wedge x = x_0 \Rightarrow P_0(x_0, x)$
- $P_f(x_0, x) \Rightarrow post(x_0, x)$
- some conditions for verification related to pairs $\ell \longrightarrow \ell'$

Asserted Program {P} S {Q}

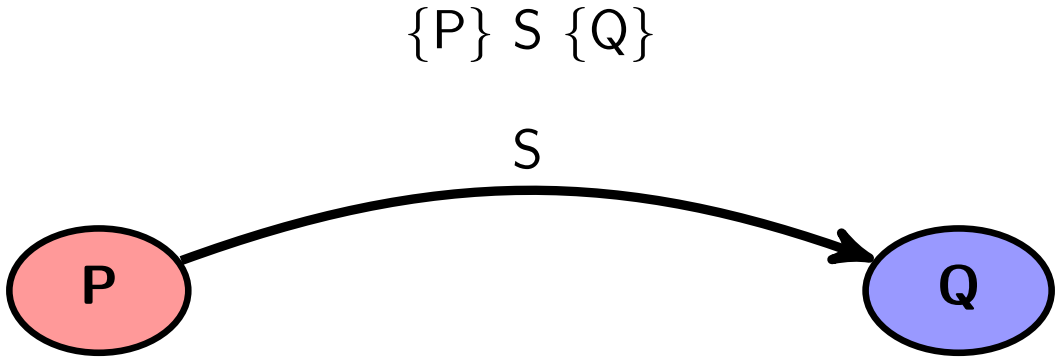


Asserted Program {P} S {Q}



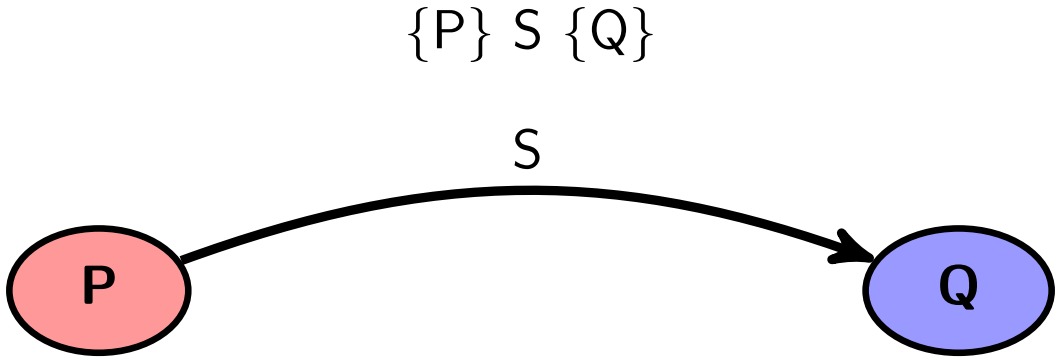
- {P} S {Q} : *asserted program*

Asserted Program {P} S {Q}



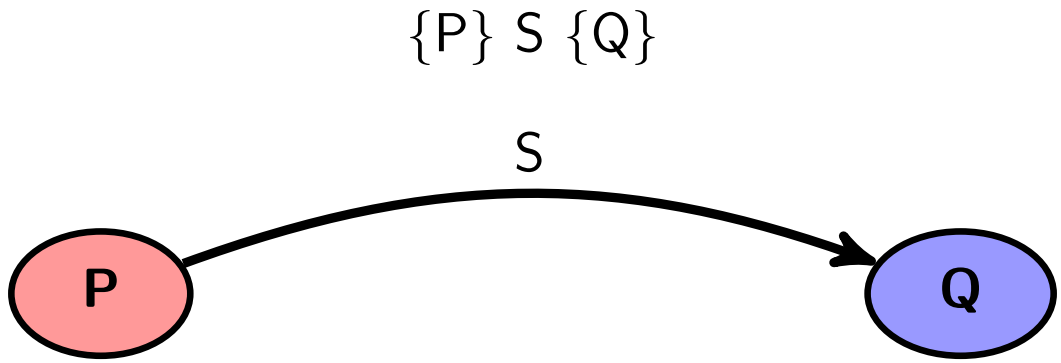
- {P} S {Q} : *asserted program*
- P $\Rightarrow WP(S)$(Q) : *logical formula*

Asserted Program {P} S {Q}



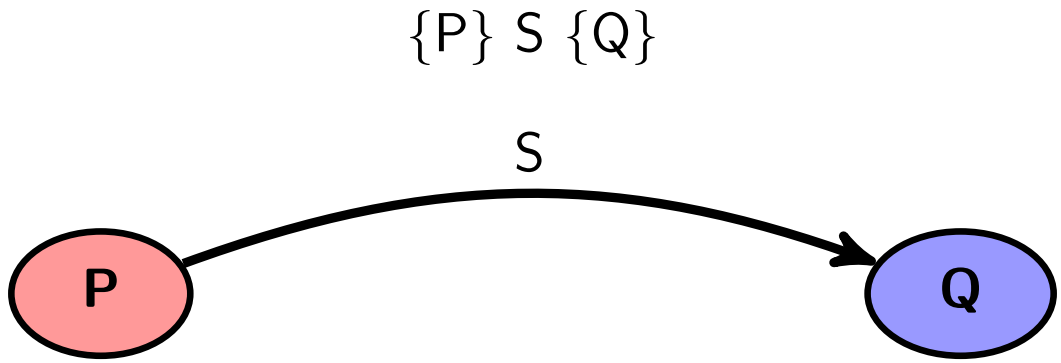
- {P} S {Q} : *asserted program*
- P $\Rightarrow WP(S)$(Q) : *logical formula*
- C(S) $\vdash$ P $\Rightarrow WP(S)$(Q) : *logical formula*

Asserted Program {P} S {Q}



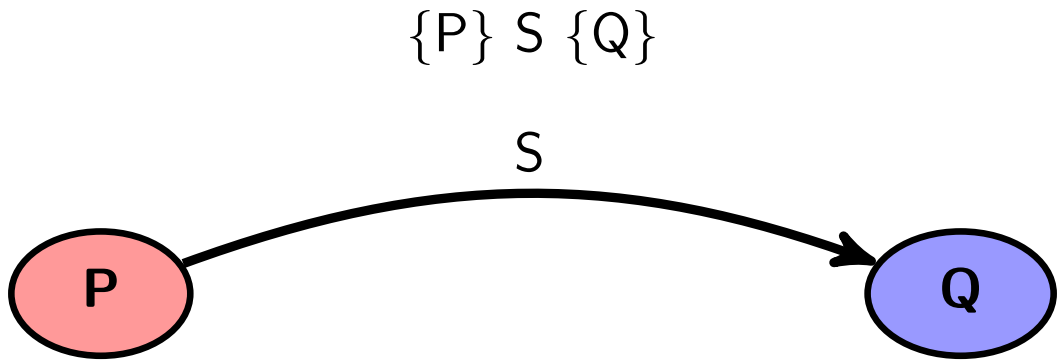
- {P} S {Q} : *asserted program*
- P $\Rightarrow WP(S)$(Q) : *logical formula*
- C(S) $\vdash$ P $\Rightarrow WP(S)$(Q) : *logical formula*
- $SP(S)$(P) $\Rightarrow$ Q : *logical formula*

Asserted Program {P} S {Q}



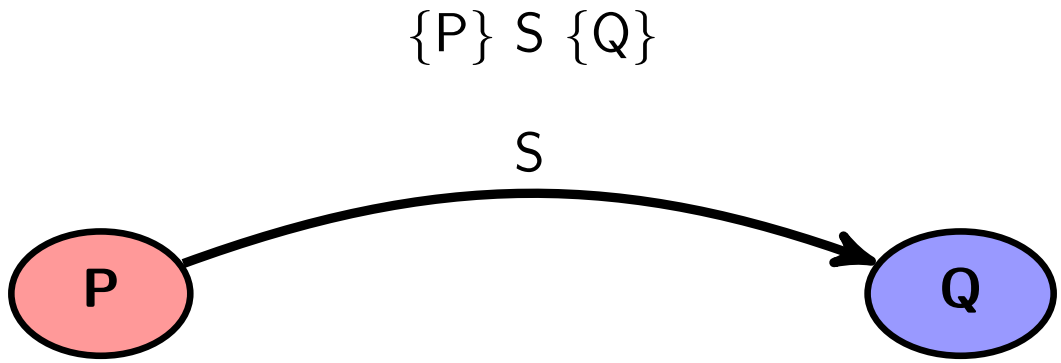
- {P} S {Q} : *asserted program*
- P $\Rightarrow WP(S)$(Q) : *logical formula*
- C(S) $\vdash$ P $\Rightarrow WP(S)$(Q) : *logical formula*
- $SP(S)$(P) $\Rightarrow$ Q : *logical formula*
- C(S) $\vdash SP(S)$(P) $\Rightarrow$ Q : *logical formula*

Asserted Program {P} S {Q}



- {P} S {Q} : *asserted program*
- P $\Rightarrow WP(S)$(Q) : *logical formula*
- C(S) $\vdash$ P $\Rightarrow WP(S)$(Q) : *logical formula*
- $SP(S)$(P) $\Rightarrow$ Q : *logical formula*
- C(S) $\vdash SP(S)$(P) $\Rightarrow$ Q : *logical formula*

Asserted Program {P} S {Q}



- {P} S {Q} : *asserted program*
- P $\Rightarrow WP(S)$(Q) : *logical formula*
- C(S) $\vdash$ P $\Rightarrow WP(S)$(Q) : *logical formula*
- $SP(S)$(P) $\Rightarrow$ Q : *logical formula*
- C(S) $\vdash SP(S)$(P) $\Rightarrow$ Q : *logical formula*

Predicate Transformer

$WP(S)$(Q) is the Weakest-Precondition of S for Q and is a predicate transformer but $WP(S)(.)$ is not a computable function over the set of predicates.

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - ☺ Vérification de propriétés de systèmes avec un analyseur Frama-c

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - ☺ Vérification de propriétés de systèmes avec un analyseur Frama-c

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - ☺ Vérification de propriétés de systèmes avec un analyseur Frama-c
- Outils

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - ☺ Vérification de propriétés de systèmes avec un analyseur Frama-c
- Outils
  - ☺ TLA/TLA$^+$ avec TLC,

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - ☺ Vérification de propriétés de systèmes avec un analyseur Frama-c
- Outils
  - ☺ TLA/TLA$^+$ avec TLC, Frama-c

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - ☺ Vérification de propriétés de systèmes avec un analyseur Frama-c
- Outils
  - ☺ TLA/TLA$^+$ avec TLC, Frama-c

Esquisse des cours, TDs et TPs

- Découpage de l'unité : 10 cours de 2 h
- Contenu :
  - ☺ Principes de modélisation des systèmes informatiques : systèmes de transition
  - ☺ Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - ☺ Modélisation de propriétés de systèmes
  - ☺ Analyse des programmes
  - ☺ Vérification de propriétés de systèmes avec un model checker TLC
  - ☺ Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - ☺ Vérification de propriétés de systèmes avec un analyseur Frama-c
- Outils
  - ☺ TLA/TLA$^+$ avec TLC, Frama-c
- Contrôle des connaissances : un écrit et un TP
  - ☺ Ecrit et TP